

Automatisation du Déploiement & renouvellement de Certificats (HTTPS/IIS) PKIs publiques

Certify Management Hub



Fiche d'Action : Migration du Déploiement de Certificats de tâche PowerShell vers une tâche de Magasin Centralisé (CCS)

Lien de dépôts : <https://github.com/BENELKAID69/Optimedit-projects>

Auteur : Optimedit IT (optimedit.fr@gmail.com)

Objectif de la Fiche

Cette fiche technique formalise la transition du mode de déploiement des certificats SSL pour la ferme de serveurs IIS sous l'infrastructure **Optimedit**. L'objectif est de remplacer la tâche du script PowerShell personnalisé de distribution à distance par une intégration native via le **Centralized Certificate Store (CCS)** piloté directement par le **Certify Management Hub**.

Objectif (actions) :

Migrer l'architecture IIS vers CCS pour :

- Supprimer le script PowerShell de déploiement
- Centraliser les certificats dans un dossier UNC
- Activer SNI sur les backends IIS
- Garder HTTP-01 pour `api-rest.optimedit.eu` (aucune modification ACME)
- Garder le déploiement automatique via Certify Hub

 Cette fiche concerne le certificat actuel (mono-domaine).

 DNS-01 n'est PAS nécessaire pour CCS dans notre cas.

Contexte et Topologie de l'Infrastructure

- **Serveur Orchestrateur & Reverse Proxy** : API-REST (192.168.1.125) hébergeant le Certify Management Hub et le stockage centralisé CCS (\\API-REST\CCSStore).
- **Serveurs Backends** : SRV-WEB-02 (192.168.1.33) et SRV-WEB-03 (192.168.1.169) configurés pour pointer dynamiquement vers le CCS.
- **Certificat Cible** : Mono-domaine `api-rest.optimedit.eu`.

■ 1. PRÉREQUIS (à faire avant toute modification)

1.1. Serveurs concernés

- Reverse-proxy : **APT-REST**
- Backends IIS : **SRV-WEB-02, SRV-WEB-03**
- Hub Certify : **API-REST**

1.2. Dossier UNC pour CCS

Sur **API-REST** (Hub), créer :

C:\CCSStore

Partage:

\\API-REST\CCSStore

Droits NTFS / Partage

- **Lecture** : SRV-WEB-02\$, SRV-WEB-03\$
- **Écriture** : compte du service Certify Hub «svc_certify» (LocalSystem ou compte dédié)

1.3. Mot de passe CCS

Choisir un mot de passe PFX **unique** pour CCS. Il sera utilisé :

- Dans IIS (SRV-WEB-02/03)
- Dans Certify Hub (task CCS)

■ 2. CONFIGURATION DU HUB CERTIFY (API-REST)

Lien site officiel : <https://docs.certifytheweb.com/docs/deployment/tasks/ccs/>

Cette tâche est assurée par le provider "Certify.Providers.DeploymentTasks.CCS", voir requête client API (Postman) à la fin de ce document.

2.1. Ajouter la tâche CCS

Dans Certify Hub → Choisir l'instance → API-REST → api-rest.optimedit.eu

→ Tasks → Deployment Tasks → **Add**:

- **Task type** : *Deploy to Centralized Certificate Store (CCS)*
- **Run on success** : Yes
- case "**Run task even if previous task step failed**" : laisser décochée
- **Path** : \\API-REST\CCSStore
- **Target Type** : Local (as current service user)

Sur Target Type : « **Local (as current service user)** »

Petite précision utile : comme le Hub tourne en LocalSystem **sur API-REST**, et que le partage \\API-REST\CCSStore est hébergé **sur cette même machine**, l'écriture se fait en pratique directement sur le disque local (C:\CCSStore), sans réel passage par une authentification réseau — donc pas de souci d'identité à ce niveau, contrairement à la lecture côté backends (qui, elle, passe bien par le réseau avec svc_certify).

Tâche Deploy-to-CCS

Edit Task	Edit Task
General Settings Task Parameters Task Type Deploy to Centralized Certificate Store (CCS) Deploy latest certificate to Windows Centralized Certificate Store. Note: If a local IIS install is present you should disable Auto deployment to avoid mixing use of local certs bindings and CCS. Edit the parameters specific to this task type under Task Parameters. Display Name Deploy-to-CCS Provide a name for this task so you can identify it later. Description Provide an optional comment to describe what this task does. Trigger Run On Success ☐ Run task even if previous task step failed	General Settings Task Parameters Target Type Local (as current service user) Additional Parameters Destination Path \\API-REST\CCSSStore UNC Path or Local Share

⚠ Important

Ne supprime pas encore ton script PowerShell. On le désactivera **après validation complète**.

■ 3. CONFIGURATION DES SERVEURS IIS BACKEND (SRV-WEB-02 / SRV-WEB-03)

3.1. Installer le rôle CCS « Web-CertProvider »

Sur chaque backend :

PowerShell

```
# ===== Installation CCS =====

$result = Install-WindowsFeature Web-CertProvider -IncludeManagementTools

# ou installation via l'iso du windows server 2022/2025
Test-path "D:\sources\sxs"
Install-WindowsFeature Web-CertProvider -IncludeManagementTools -Source
"D:\sources\sxs"

# ===== vérification de l'installation CCS =====

$result | Format-List *
Get-WindowsFeature Web-Mgmt-Tools
Get-WindowsFeature Web-Mgmt-Console
Get-WindowsFeature Web-CertProvider
```

Ou installations avec un script vers plusieurs serveurs backends

```
$servers = @("SRV-WEB-02", "SRV-WEB-03")
$cred = Get-Credential

foreach ($srv in $servers) {
    Invoke-Command -ComputerName $srv -Credential $cred -ScriptBlock {
        Install-WindowsFeature Web-CertProvider -IncludeManagementTools
    }
}
Invoke-Command -ComputerName SRV-WEB-02 -ScriptBlock {
    Get-WindowsFeature Web-CertProvider
}

Invoke-Command -ComputerName SRV-WEB-03 -ScriptBlock {
```

```
} Get-WindowsFeature Web-CertProvider
```

Résultat :

Display Name: Centralized SSL Certificate Support
Installed: True

3.2. Activer CCS avec PowerShell ou via IIS Manager

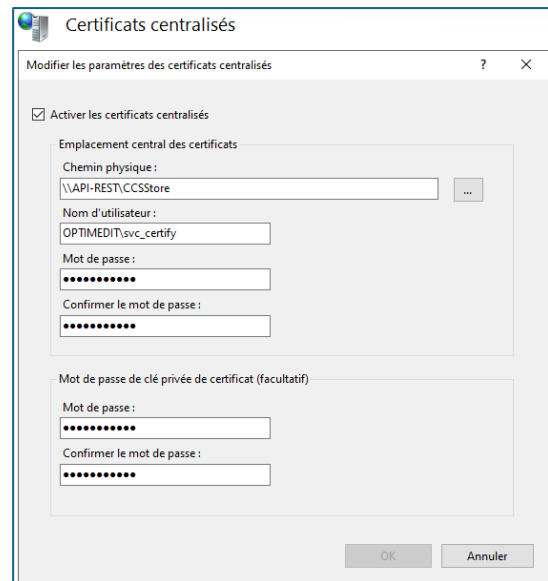
3.2.1. Activer CCS avec PowerShell

```
# ===== Configuration =====  
  
# compte de service = OPTIMEDIT\svc_certify + Mot de passe  
$readCred = Get-Credential -Message "Compte avec accès lecture au partage CCS"  
  
Enable-IISCentralCertProvider `   
-CertStoreLocation "\\API-REST\CCSStore" `   
-UserName $readCred.UserName `   
-Password $readCred.Password `   
-PrivateKeyPassword (Read-Host -AsSecureString "Mot de passe privé CCS")  
  
# ===== Vérification de conf =====  
  
Get-IISCentralCertProvider  
  
Enabled=True  
CertStoreLocation=\\API-REST\CCSStore  
UserName=OPTIMEDIT\svc_certify  
Password=*****  
PrivateKeyPassword=*****
```

3.2.2 Activer CCS via IIS Manager

IIS Manager → Serveur → **Centralized
Certificates/Certificats centralisés** :

- **Enable CCS/Activer les certificats centralisés**
- **Path/Chemin physique** : \\API-REST\CCSStore
- **Nom d'utilisateur** : compte lecture
- **Mot de passe** : mot de passe CCS
- **Private Key Password/Mot de passe de clé privée** : mot de passe CCS



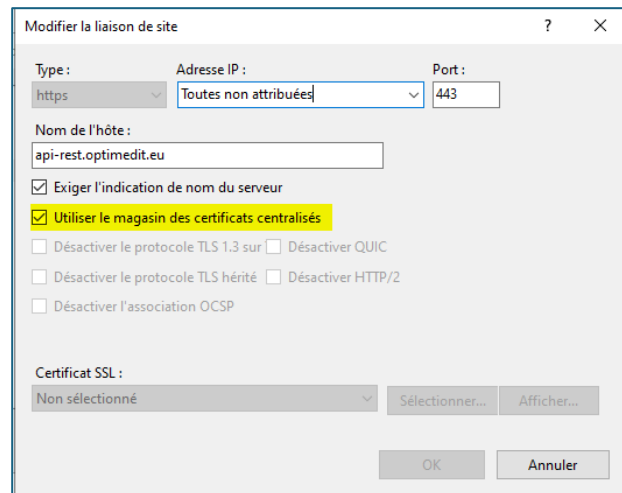
```
# ===== Vérification de configuration =====  
  
Get-IISCentralCertProvider  
  
Enabled=True  
CertStoreLocation=\\API-REST\CCSStore  
UserName=OPTIMEDIT\svc_certify  
Password=*****  
PrivateKeyPassword=*****
```

Maintenant que le moteur CCS est actif sur les backends, il faut faire pointer le site vers lui dans « bindings »

3.3. Activer SNI sur les bindings HTTPS

Sur chaque backend :

1. Supprimer l'ancien binding « `https *:443` » (non-SNI) (celui du script PowerShell)
2. Ajouter un binding HTTPS :
 - **Type** : `https`
 - **Port** : `443`
 - **Host** : `api-rest.optimedit.eu`
 - **SNI** : Enabled
 - **Certificat** : Centralized Certificate Store



C'est normal que le champ "**Certificat SSL**" soit grisé dans la fenêtre de liaison HTTPS dès que on coche "Utiliser le magasin des certificats centralisés".

4. CONFIGURATION DU REVERSE-PROXY (APT-REST)

APT-REST doit :

- Accepter SNI
- Router `api-rest.optimedit.eu` vers SRV-WEB-02/03
- Ne rien changer pour HTTP-01 (port 80)

Verification

Test:

```
Invoke-WebRequest http://api-rest.optimedit.eu → 200 OK  
Invoke-WebRequest https://api-rest.optimedit.eu → 200 OK
```

➡ Reverse-proxy déjà correct ➡ Rien à modifier pour CCS

■ 5. TEST DE MIGRATION CCS

Étapes de test dans l'ordre

5.1. Déclencher un renouvellement réel

Dans le Hub, sur `api-rest.optimedit.eu` → **Request Certificate**.

5.2. Vérifier les logs du Hub – confirmer l'exécution de la tâche

Vous devriez voir les **trois** tâches s'exécuter en séquence :

```
Task [latest.pfx] :: Task Completed OK          (export PFX, inchangé)
Task [Deploy-cert-to-backends-pwsh] :: Task Completed OK  (ancien script, toujours actif)
Task [Deploy-to-CCS] :: Task Completed OK      (nouvelle tâche CCS)
```

Logs:

2026-07-23 23:43	[INF]	latest.pfx :: Task Completed OK
2026-07-23 23:43	[INF]	Deploy-cert-to-backends-pwsh :: Task Completed OK
2026-07-23 23:43	[INF]	Deploy-to-CCS :: Task Completed OK

5.3. Vérifier que le fichier <hostname>.pfx est bien apparu dans le partage

```
PowerShell
Get-ChildItem "C:\CCSStore\"
```

Vérifiez le **nom exact** du fichier généré — c'est le point le plus susceptible de poser problème (convention de nommage `<hostname>.pfx` attendue par CCS).

(Get-ChildItem "C:\CCSStore\").Name api-rest.optimedit.eu.pfx
--

5.4. Vérifier que les serveurs backends servent bien ce certificat via CCS

Vérification de la source réelle du certificat, pas juste sa valeur :

```
Get-WebBinding -Name "Default Web Site" -Protocol https | Format-List
```

- **sslFlags** contient la valeur 2 ou 3 (bit CCS actif) ET **certificateHash** est vide ou non présent → **binding réellement servi par CCS.**
- **certificateHash** contient une valeur (le thumbprint) → le binding utilise encore le **magasin local classique, pas CCS**, même si le certificat est le bon.

Résultat de déploiement non réussi :

```
SRV-WEB-02
PS C:\Users\Administrateur.OPTIMEDIT> Get-WebBinding -Name "Default Web Site" -Protocol https | Select-Object bindingInformation, sslFlags, certificateHash
bindingInformation sslFlags certificateHash
-----
*:443:              0 EC843D938E4C974B6636AA8C609EDD1F33355583
```

Microsoft définit sslFlags comme un **bitmask** (drapeaux combinables) :

Valeur	Nom officiel Microsoft	Signification
0	None	Binding HTTPS classique (certificat local non CCS)
1	Sni	SNI activé
2	CentralCertStore	CCS activé
3	Sni + CentralCertStore	SNI + CCS activés

Explication de la valeur 3 attendue

Dans l'API .NET utilisée par IIS :

Namespace : `Microsoft.Web.Administration` Classe : `Binding` Propriété : `SslFlags`

Microsoft définit :

[Flags]

```
public enum SslFlags {
```

```
    None = 0,
```

```
    Sni = 1,
```

```
    CentralCertStore = 2
```

```
}
```

Donc

Sni (1) + CentralCertStore (2) = 3

<https://learn.microsoft.com/en-us/iis/configuration/system.applicationhost/sites/site/bindings/binding>

Si « sslFlags » retour 3 c'est Ok, sinon analyser le déploiement

Analyse de résultat de « Get-WebBinding » avec la valeur 0

Ce n'est PAS un binding CCS — un vrai binding CCS aurait certificateHash **vide ou non présent** (le certificat n'est jamais copié localement, seulement lu depuis le partage à la volée) et un flag SSL spécifique activé 2 ou 3. Ici, certificateHash contient bel et bien un thumbprint : le binding pointe encore vers le magasin local classique.

Le vrai problème : les deux tâches se battent pour le même binding

Pour corriger :

1. Comme dans l'étape « 3.3. Activer SNI sur les bindings HTTPS » Reconfigurer à nouveau le binding CCS sur les SRV-WEB-02/03 (puisque le dernier renouvellement l'a écrasé avec la tâche PowerShell).

2. Seulement maintenant, désactiver (pas supprimer) la tâche PowerShell dans le Hub. Pour désactiver la tâche utiliser son « General Setting », « Trigger », Disabled (Will Not Run), Ok , Save.

3. Relancer la commande précédente sur les deux serveurs backends :
Get-WebBinding -Name "Default Web Site" -Protocol https | Format-List

Où le script de vérification (script suivant [Test-ccs-multi-serveurs.ps1](#)) — vous devriez cette fois obtenir le flag CCS actif avec la valeur 3, sans qu'aucune tâche ne vienne l'écraser au prochain cycle.

4. Faire un dernier test de renouvellement (Request Certificate) pour confirmer que, PowerShell désactivé, le binding CCS **reste stable** après ce cycle — c'est la vraie preuve finale.

Résultats de déploiement réussi sur les serveurs backends:

```
PS C:\Users\Administrateur.OPTIMEDIT> hostname
SRV-WEB-02
PS C:\Users\Administrateur.OPTIMEDIT> Get-WebBinding -Name "Default Web Site" -Protocol https | Format-List

protocol           : https
bindingInformation : *:443:api-rest.optimedit.eu
sslFlags           : 3

PS C:\Users\Administrateur.OPTIMEDIT> hostname
SRV-WEB-03
PS C:\Users\Administrateur.OPTIMEDIT> Get-WebBinding -Name "Default Web Site" -Protocol https | Format-List

protocol           : https
bindingInformation : *:443:api-rest.optimedit.eu
sslFlags           : 3
```

Test consolidé multi-serveurs à distance après correction sur API-REST:

```
# Test-ccs-multi-serveurs.ps1
$servers = @("SRV-WEB-02", "SRV-WEB-03")
$expectedThumbprint = "EC843D938E4C974B6636AABC609EDD1F33355B3"

write-Host "`n== 1. Vérification de la SOURCE du certificat (binding IIS) == " -
ForegroundColor Cyan
foreach ($srv in $servers) {
    Invoke-Command -ComputerName $srv -ScriptBlock {
        $binding = Get-WebBinding -Name "Default Web Site" -Protocol https
        $ccs = Get-IISCentralCertProvider

        $sourceCCS = ($binding.sslFlags -band 2) -ne 0 -and
        [string]::IsNullOrEmpty($binding.certificateHash)

        [PSCustomObject]@{
            Serveur          = $env:COMPUTERNAME
            BindingInfo      = $binding.bindingInformation
            SslFlags         = $binding.sslFlags
            CertHashLocal    = if ($binding.certificateHash) { $binding.certificateHash }
        }
        else { "(vide)" }
        CCS_Enabled         = $ccs.Enabled
        SourceReelle        = if ($sourceCCS) { "☑ CCS" } else { "⚠ Magasin local (pas CCS)" }
    }
}

write-Host "`n== 2. Vérification du certificat RÉELLEMENT présenté (TLS réseau) == " -
ForegroundColor Cyan
$urls = @(
    "https://api-rest.optimedit.eu"
    #"https://srv-web-02.optimedit.eu", # après migration de CCS url ne fonctionnera pas car SNI activé
    #"https://srv-web-03.optimedit.eu", # après migration de CCS url ne fonctionnera pas car SNI activé
)
foreach ($url in $urls) {
    try {
        $request = [System.Net.HttpWebRequest]::Create($url)
        $request.ServerCertificateValidationCallback = { $true }
        $request.GetResponse() | Out-Null
        $thumb = $request.ServicePoint.Certificate.GetCertHashString()
        $match = if ($thumb -eq $expectedThumbprint) { "☑ Conforme" } else { "✗ DIFFÉRENT" }

        [PSCustomObject]@{
            URL          = $url
            Thumbprint    = $thumb
            Statut        = $match
        }
    }
    catch {
        write-Host "Échec sur $url : $($_.Exception.Message)" -ForegroundColor Red
    }
}
```



```
=== 1. Vérification de la SOURCE du certificat (binding IIS) ===

Serveur      : SRV-WEB-02
BindingInfo  : *:443:api-rest.optimedit.eu
SslFlags     : 3
CertHashLocal : (vide)
CCS_Enabled  : 
SourceReelle : ☒ CCS
PSCoordinateName : SRV-WEB-02
RunspaceId   : d966b826-a4e0-4b15-9e0d-d7ce54851f47

Serveur      : SRV-WEB-03
BindingInfo  : *:443:api-rest.optimedit.eu
SslFlags     : 3
CertHashLocal : (vide)
CCS_Enabled  : 
SourceReelle : ☒ CCS
PSCoordinateName : SRV-WEB-03
RunspaceId   : ec0ba4bf-782d-4746-a86d-246048307459

=== 2. Vérification du certificat RÉELLEMENT présenté (TLS réseau) ===
URL          : https://api-rest.optimedit.eu
Thumbprint   : EC843D938E4C974B6636AABC609EDD1F333555B3
Statut       : ☒ Conforme
```

Si message "⚠ Magasin local (pas CCS)" Reprenez exactement la même séquence

1. Reconfigurer le binding HTTPS pour pointer vers "**Centralized Certificate Store**"
Comme dans l'étape « **3.3. Activer SNI sur les bindings HTTPS** »
2. Désactiver la tâche de PowerShell dans les Setting
3. Relancer le script de vérification ci-dessus.

■ 6. MIGRATION FINALE (désactivation du script PowerShell)

Une fois :

- CCS OK
- SNI OK
- Reverse-proxy OK
- Backends OK
- Certificat servi correctement par CCS

Alors :

6.1. Supprimer la tâche PowerShell dans Certify Hub

Supprimer la tâche PowerShell et faire un renouvellement (Request Certificate) pour confirmer que, PowerShell n'est plus sur l'infra, le binding CCS **reste stable** après ce cycle — c'est la vraie preuve finale de cette migration vers CCS.

Managed Certificate → Tasks → Delete :

Deploy-cert-to-backends-pwsh

6.2. Supprimer le script PowerShell du serveur

C:\CertifyScripts\pwsh\deploy-cert-to-backends-pwsh-7.ps1

Le script n'est plus utilisé.

7. SCHÉMA — Pipeline final CCS + SNI + HTTP-01

ACME VALIDATION (HTTP-01)

Let's Encrypt → `http://api-rest.optimedit.eu/.well-known/...` → **API-REST** (Reverse-Proxy + Hub)
- Port 80

CERTIFICAT ÉMIS

Certify Hub → Génère un PFX interne
Source réelle : `C:\ProgramData\certify\assets\api-rest.optimedit.eu\*.pfx`
(Ce PFX est en mémoire et sur disque)

EXPORT LOCAL (laisse pour troubleshooting)

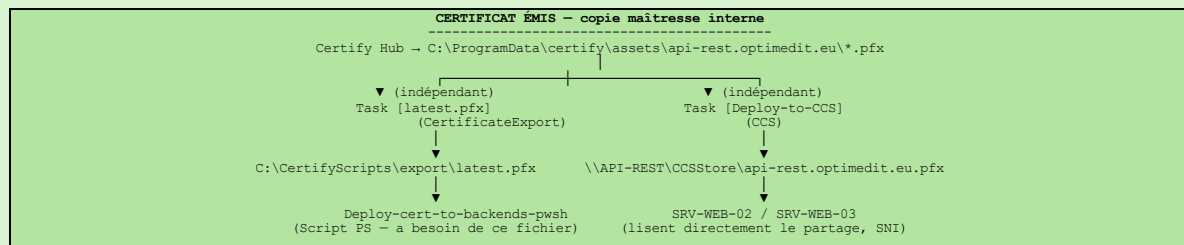
Task [latest.pfx] → `C:\CertifyScripts\export\latest.pfx`
- indépendant
- n'est pas utilisé par CCS
- n'est pas lu par d'autres tâches
uniquement pour l'ancienne tâche du script PowerShell

DISTRIBUTION VIA CCS (HTTPS)

Task [Deploy-to-CCS]
- copie DIRECTEMENT le PFX interne
- source : `C:\ProgramData\certify\assets\api-rest.optimedit.eu\*.pfx`
- destination : `\\API-REST\CCSStore\api-rest.optimedit.eu.pfx`

APT-REST (SNI ON)

↓
SRV-WEB-02 (SNI ON → CCS) SslFlags 3
SRV-WEB-03 (SNI ON → CCS) SslFlags 3



8. RÉSUMÉ FINAL (fiable et validé)

- ✓ CCS impose SNI
- ✓ SNI impose des bindings HTTPS par domaine
- ✓ HTTP-01 reste inchangé (port 80)
- ✓ DNS-01 n'est pas nécessaire pour CCS
- ✓ On peut migrer vers CCS sans toucher à ACME
- ✓ On peut supprimer ton script PowerShell après validation
- ✓ On peut garder ton certificat mono-domaine tel quel

Requête GET client API (Postman) pour visualiser le provider utilisé dans ce déploiement et les détails de la tâche créée «Deploy-to-CCS»

Les variables :

hub_api_url = https://api-rest.optimedit.eu:8443

access_token = sortie de requête 'POST {{hub_api_url}}/api/v1/auth/login' = "accessToken": "eyJ...DIU"

instanceId = 17dfbdd7-b834-4da0-8f3c-e18f8395b48a

managedCertId = f9712ad8-bd49-430c-8cff-370633369e53

format = pfx

Requête

GET {{hub_api_url}}/internal/v1/certificate/{{instanceId}}/settings/{{managedCertId}}

Optimedit API > Renouvellement cert > 02.Test-configuration

GET {{hub_api_url}}/internal/v1/certificate/{{instanceId}}/settings/{{managedCertId}}

Auth Type: Inherit auth from parent

Bearer Token: Token: {{access_token}}

Body: Cookies Headers 8 Test Results 200 OK - 8 ms

```
{
  "id": "c152593d-ce25-4d99-833d-78612c8965da",
  "taskTypeId": "Certify.Providers.DeploymentTasks.CCS",
  "taskName": "Deploy-to-CCS",
  "description": "",
  "isFatalOnError": false,
  "retriesAllowed": 0,
  "retryDelaySeconds": 10,
  "challengeProvider": "Certify.StandardChallenges.Local",
  "challengeCredentialKey": "",
  "targetHost": "",
  "parameters": [
    {
      "key": "path",
      "value": "\\\\API-REST\\CCSStore"
    }
  ],
  "dateLastExecuted": "2026-07-24T14:11:49.7787042+00:00",
  "lastResult": "Task Completed OK",
  "lastRunStatus": 3,
  "taskTrigger": 2,
  "runIfLastStepFailed": false,
  "isChanged": false
},
{
  "id": "f9712ad8-bd49-430c-8cff-370633369e53",
  "version": 54,
  "groupId": "",
  "serverSiteId": "",
  "instanceId": "17dfbdd7-b834-4da0-8f3c-e18f8395b48a",
  "externalSource": null,
  "name": "api-rest.optimedit.eu",
  "comments": null,
  "itemType": 1,
  "dateStart": "2026-07-24T15:13:15+02:00",
  "dateExpiry": "2026-10-22T15:13:14+02:00",
  "dateRenewed": "2026-07-24T14:11:48.5716908+00:00",
```